

Draw It Or Lose It

CS 230 Project Software Design Template
Version 2.0

Table of Contents

[CS 230 Project Software Design Template](#) 1

[Table of Contents](#) 2

[Document Revision History](#) 2

[Executive Summary](#) 3

[Requirements](#) 3

[Design Constraints](#) 5

[System Architecture View](#) 5

[Domain Model](#) 5

[Evaluation](#) 7

[Recommendations](#) 10

Document Revision History

Version	Date	Author	Comments
1.0	09/20/2024	Vonds Auguste	Original version
2.0	10/05/2024	Vonds Auguste	Updated UML & Evaluation

Executive Summary

The Gaming Room has tasked us with developing a web-based version of their game, *Draw It or Lose It*, a team-based drawing game where teams guess puzzles from rendered images. The goal is to develop the software application by adhering to the provided software requirements while preparing for future hardware decisions. Key requirements include ensuring that the game supports multiple teams, each with multiple players, unique team and game names, and the existence of only one game instance in memory at any given time. By implementing object-oriented principles and design patterns, we will ensure the application is scalable, maintainable, and efficient for deployment in a distributed environment.

This document outlines the software design for the game application. The primary focus will be implementing the **singleton pattern** to manage game instances and the **iterator pattern** to handle game, team, and player creation efficiently. These design choices will streamline development, ensuring scalability and ease of management as the application grows.

Requirements

Business Requirements

Performance:

The game should render images smoothly without delays or lags. Response times for submitting guesses or checking for name availability must be quick to ensure seamless gameplay.

Scalability:

The app must be able to handle multiple concurrent game sessions with multiple teams and players. The backend should support scaling to handle a growing number of users as the game gains popularity.

Availability:

The web-based version should be available 24/7 with minimal downtime. A reliable hosting infrastructure should be used to ensure uptime and availability for players globally.

Security:

Players should be authenticated securely. Sensitive data, such as usernames and game data, must be encrypted during transmission and storage. The system should ensure that only authorized users can create and join games, and team names should be protected from duplication.

Usability:

The user interface must be intuitive and easy to use for both new and returning players. The game must be accessible to users with disabilities, following best practices for web accessibility.

Portability:

The game must run smoothly across different platforms, including web browsers and mobile operating systems. The design should be responsive to adapt to different screen sizes and orientations (desktop, tablet, smartphone).

Maintainability:

The codebase should follow modular and clean coding practices to facilitate future updates and maintenance. Documentation must be provided to allow future developers to understand and extend the codebase easily.

Technological Requirements

Backend:

A server-side component to handle game logic, state management, and multi-player functionality. Technologies such as Node.js, Java, or Python for backend development.

Frontend:

A responsive frontend for cross-platform support. Technologies such as React, Vue.js, or Angular for building a dynamic user interface.

Database:

A database to store game data, player profiles, scores, and game history. Relational databases (like MySQL or PostgreSQL) or NoSQL solutions (like MongoDB) can be used based on the scalability requirements.

API:

A set of RESTful or GraphQL APIs for communication between the frontend and backend components, as well as for future mobile integration.

Design Constraints

1. **Web-Based Distributed Environment:** The game will be deployed in a distributed, web-based environment, meaning that we need to account for multiple users accessing the application concurrently. This necessitates careful management of state and memory to prevent conflicts or redundant instances of games, teams, or players.
2. **Singleton Design Pattern:** To ensure that only one instance of the game exists at any given time, the **singleton pattern** will be used to manage the `GameService` class. This will ensure efficient memory usage and prevent conflicts during game creation and management.
3. **Unique Names:** Team and game names must be unique across all instances. This will require implementing validation using the **iterator pattern** to search existing entities before allowing new ones to be created. This design constraint ensures that there are no naming conflicts within the system.
4. **Scalability:** The game should be designed to handle multiple teams and players efficiently. This means that object-oriented principles such as inheritance, polymorphism, and encapsulation must be used to allow future enhancements without major refactoring.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

Game to Team: A game can have multiple teams. This is a *one-to-many relationship*.

- One Game has many Teams.

Team to Player: A team consists of multiple players. This is a *one-to-many relationship*.

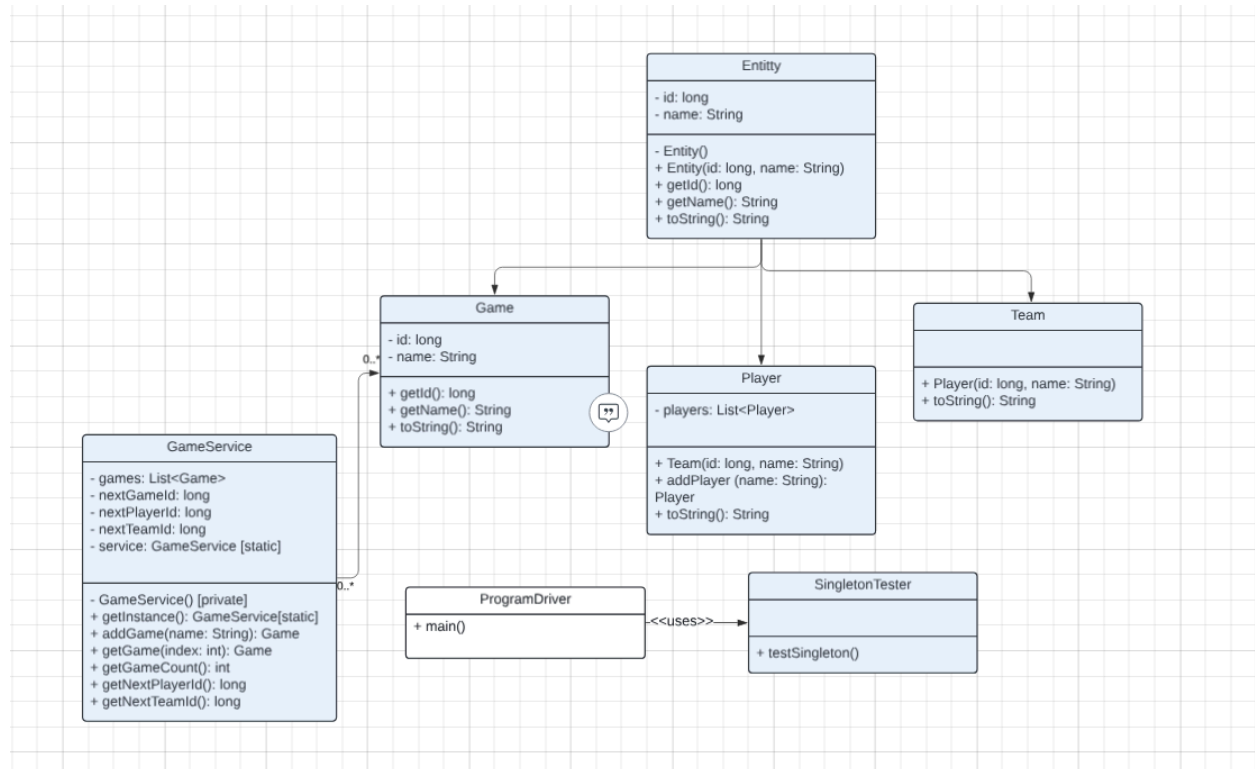
- One Team has many Players.

Game to Round: A game consists of multiple rounds (typically four). This is a *one-to-many relationship*.

- One Game has many Rounds.

Round to Puzzle: A round involves one puzzle. This is a *one-to-one relationship*.

- One Round has one Puzzle.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Characteristics: MacOS is UNIX-based, offering a stable environment for web hosting and development with native support for many web hosting tools (e.g., Apache, Nginx). Mac supports development tools like Xcode and Homebrew for managing dependencies. Advantages: Secure and reliable due to its UNIX foundation. Great for development and testing in cross-platform environments (iOS, Linux).	Characteristics: Linux is widely used in server environments due to its flexibility, open-source nature, and low cost. It supports most web technologies like Apache, Nginx, MySQL, and PHP. Advantages: Cost-effective, with a variety of free distributions (Ubuntu, CentOS). Excellent for scalability, customization, and optimization. High availability of server-side software and hosting tools. Extensive support and large community for troubleshooting	Characteristics: Windows Server supports IIS (Internet Information Services) and has strong integration with Microsoft products like .NET, SQL Server, and Azure. It's more common in enterprise-level hosting environments. Advantages: Excellent support for Microsoft technologies like ASP.NET, .NET Core, and C#. User-friendly interface and administrative tools (GUI-based). Ideal for businesses that use Microsoft enterprise software. Weaknesses:	Characteristics: Mobile devices typically aren't used to host web applications, but mobile platforms can run lightweight servers for development and testing (e.g., local servers on Android or iOS). Advantages: Suitable for local development environments or testing mobile app services. Handy for lightweight services and mobile web app testing. Tools like Ngrok or Termux on Android allow basic server capabilities. Weaknesses:

	Well-suited for web development with built-in access to terminal and scripting tools. Weaknesses: High cost of Mac hardware compared to Linux-based servers. Rarely used in production environments for large-scale web hosting. Limited vendor support for commercial web hosting services.	and improvements. Weaknesses: Steeper learning curve for new users unfamiliar with command-line tools. Less support for certain proprietary applications compared to Windows.	Higher licensing costs compared to Linux. Lower performance and efficiency than Linux for certain high-demand web services.	Limited processing power and storage compared to traditional servers. Not suitable for hosting high-traffic applications or services. Not designed for continuous, long-term hosting.
Client Side	Cost: Development for Mac often requires purchasing Apple hardware, which can be expensive. Mac development environments (like Xcode) are	Cost: Linux is free and open-source, making it cost-effective. Many development tools are also free and community-supported.	Cost: Windows development environments (like Visual Studio) are available for free, but enterprise versions or Windows Server licenses can incur costs.	Cost: Mobile development typically requires specialized tools for each platform (Xcode for iOS, Android Studio for Android). The cost of devices and developer licenses (like Apple's

	free but require macOS. Time: Learning Xcode and Swift for macOS/iOS development can be time-consuming. However, macOS supports cross-platform tools that can streamline multi-platform development. Expertise: Developers need expertise in macOS-specific frameworks (Swift, Objective-C) and cross-platform tools like Electron or React Native for supporting multiple clients.	Time: Development on Linux may take longer due to the steeper learning curve of setting up environments and troubleshooting compatibility issues across distributions. Expertise: Linux developers need familiarity with terminal-based development, package management, and various open-source development tools. Expertise in cross-platform development frameworks (like Flutter) is useful for supporting multiple clients.	Time: Windows is user-friendly, making it easier to set up environments quickly. It also integrates well with Microsoft technologies, streamlining development time for specific use cases. Expertise: Developers need expertise in C#, .NET, and related technologies for Windows-based development. Additionally, knowledge of cross-platform frameworks (like Xamarin) is helpful.	developer program) can add up. Time: Mobile development often requires additional time to adapt software to different platforms (iOS, Android) and devices (tablets, phones). Expertise: Developers need proficiency in Swift for iOS and Kotlin/Java for Android. Cross-platform frameworks (Flutter, React Native) reduce development time by supporting multiple platforms with a single codebase. >
Development Tools	Languages: Swift, Objective-C, Python, Ruby, PHP	Languages: Python, Java, PHP, C/C++	Languages: C#, C++, .NET, Python, Java IDEs: Visual Studio, JetBrains Rider	Languages: Swift (iOS), Kotlin/Java (Android), Dart (Flutter), JavaScript (React Native)

	IDEs: Xcode, Visual Studio Code Other Tools: Homebrew (package management), Docker, Terminal (for UNIX-based development) Mac is highly suitable for iOS and macOS development but also supports cross-platform development.	IDEs: Eclipse, VS Code, Sublime Text Other Tools: Docker, Git, Jenkins, Ansible Linux is favored for server-side applications, DevOps, and backend development.	Other Tools: PowerShell, Docker, Git for Windows Ideal for enterprise and .NET development but can be used for other technologies as well.	IDEs: Android Studio, Xcode, Visual Studio Code Other Tools: Emulator tools for testing across devices, Firebase, Postman (for API testing).
--	---	--	---	--

Recommendations

To expand *Draw It or Lose It* into a multi-platform, web-based game, it is essential to evaluate and choose the appropriate system architecture, operating platform, storage, memory management, and security considerations. Below is a detailed analysis and recommendation.

Operating Platform Recommendation

I recommend using a **Linux-based operating platform** for *The Gaming Room* to expand *Draw It or Lose It*. Linux is a powerful and flexible platform that supports **cross-platform development**, scalability, and high availability, making it ideal for a web-based game designed to run on multiple platforms (web, mobile, etc.).

Scalability: Linux supports both vertical and horizontal scaling, allowing the game to handle a growing user base.

Cost-Effective: Linux is open-source, meaning no licensing fees, which is an advantage compared to platforms like Windows.

Compatibility: Linux is compatible with various server environments (e.g., Apache, Nginx) and can easily integrate with other systems such as Android (since Android is based on the Linux kernel).

Cross-Platform Development: Linux supports a variety of development tools (Node.js, Python, Java, etc.), allowing for seamless development across multiple environments.

Operating System Architectures

The Linux platform architecture offers several advantages for hosting *Draw It or Lose It*:

Modular Design: Linux operating systems are modular and lightweight, allowing you to optimize the environment by selecting only the components needed for the game application.

Multitasking: Linux supports multi-user, multitasking environments, allowing *Draw It or Lose It* to handle multiple concurrent connections and game sessions.

Kernel Customization: The Linux kernel can be customized to suit specific needs, which allows developers to tweak performance and security settings specifically for hosting the game.

High Availability: Linux provides built-in support for high availability and fault-tolerant systems through clustering and load balancing techniques.

Storage Management

An appropriate storage management system for the Linux platform would be **Amazon Web Services (AWS) S3** or **Google Cloud Storage**. Both systems are highly scalable, reliable, and provide cost-effective storage options that are well-suited for storing large volumes of data, such as:

User Data: Information about players, game scores, and progress can be stored securely.

Game Assets: Stock images, game state data, and session records can be efficiently stored and retrieved.

Automatic Scaling: Both AWS S3 and Google Cloud offer automatic scaling and high availability, ensuring that the game can handle increasing numbers of users without latency issues.

Other Advantages:

Backup and Redundancy: Cloud storage systems ensure data redundancy and protection against data loss.

Security: These systems offer encryption at rest and in transit, which is vital for protecting user data.

Memory Management

In Linux, **memory management** is handled through various efficient techniques that are ideal for a multi-platform game like *Draw It or Lose It*:

Virtual Memory: Linux makes extensive use of virtual memory, where inactive memory pages are moved to disk (swap space), freeing up RAM for active processes. This helps maintain performance, even with high usage.

Shared Memory: Linux supports shared memory, which allows multiple processes to access the same memory space. This is useful when sharing assets (e.g., game images) between different processes in the game.

Cache Management: Linux uses an intelligent file system cache to improve disk I/O performance, reducing the load on memory and disk when accessing frequently used data like game images or player profiles.

Garbage Collection: For the *Draw It or Lose It* software, garbage collection in managed languages like Java (if used in the application) will automatically manage the cleanup of unused objects, ensuring efficient memory use.

Benefit for the Game: Efficient memory management ensures that the game runs smoothly, even with multiple users, by optimizing how memory is allocated and deallocated during the game session.

Distributed Systems and Networks

To allow *Draw It or Lose It* to communicate between various platforms (web, mobile, desktop), a **distributed system architecture** can be implemented using technologies like **REST APIs** or **WebSockets** over HTTP/HTTPS:

RESTful APIs: The game backend can expose RESTful endpoints to handle game state changes, team creation, and player guesses. These APIs can be accessed from any platform (web, Android, iOS), ensuring platform independence.

Client-server communication will be asynchronous, allowing multiple platforms to interact with the same game session.

WebSockets for Real-Time Interaction: WebSockets enable two-way communication between the server and the clients in real-time, which is ideal for features like drawing updates and live player guesses.

This allows players to receive real-time updates, such as the drawing progress and other teams' guesses.

Distributed Components: The system can be designed in a **microservices architecture**, where different services (game logic, authentication, leaderboard, etc.) are deployed and scaled independently.

This ensures better maintainability and scalability, with components working seamlessly across multiple platforms.

Handling Outages and Connectivity Issues:

Fallback mechanisms and **retry policies** should be implemented to handle connectivity issues. If a client loses connection during gameplay, the game state should be saved and restored when the connection is re-established.

The **load balancer** can distribute requests across multiple instances of the game backend to ensure that downtime or server failures do not affect the user experience.

Security

Since security is a must-have for *The Gaming Room*, several security techniques need to be employed to protect user information across various platforms:

Data Encryption:

Encryption in Transit: All communication between clients (players) and the server should be encrypted using **HTTPS** (SSL/TLS).

Encryption at Rest: User data, game state, and other sensitive information should be encrypted while stored in the database or cloud storage (e.g., AWS S3, Google Cloud).

Authentication and Authorization:

Implement **OAuth 2.0** or **JWT (JSON Web Tokens)** for user authentication to ensure secure access across platforms.

User accounts and credentials should be securely managed and never stored in plain text. Passwords should be hashed and salted using secure algorithms such as **bcrypt**.

Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF) Protection:

Use security headers (e.g., **Content Security Policy (CSP)**) and input validation to protect the game's web interface from **XSS** attacks.

CSRF tokens should be used to ensure that unauthorized requests cannot be made on behalf of a user.

DDoS Protection:

Distributed Denial of Service (DDoS) attacks can be mitigated using **Cloudflare** or similar services that provide DDoS protection by filtering out malicious traffic before it reaches the game servers.

Role-based Access Control (RBAC):

Implement role-based access control to manage what each type of user (e.g., player, admin, moderator) can access within the game system.

Monitoring and Auditing:

Implement **logging and monitoring** systems (e.g., AWS CloudWatch, Prometheus) to track access logs, detect anomalies, and audit suspicious activities. Alerts can be configured to respond to potential security breaches.